Pennywise: Learning to Route LLM Calls with an Open Decision Model

Policy-Gradient Training of Laya as a Cost-Aware Router

Pennywise contributors

Built 2026-10-26 from commit `a0644e5-dirty` · seed 0

Interim draft. Built from real outcomes while RL training is still running. Only the RL policies for $\lambda \in \{0\}$ had finished, so the mode selection, the RL rows and the Pareto curve cover those values only. Every number will change when the remaining runs finish. The policy-map figure is omitted until the visualisation export runs.

Abstract. Agents send every request to a frontier model, including the many bounded ones a far cheaper model would answer correctly. We treat model-tier selection as a contextual bandit and use an open-weight non-autoregressive decision model, Laya [1], as the policy: its option distribution over {small, medium, large} *is* $\pi(a|s)$, available from a single forward pass. Starting from a supervised warm start on hindsight-optimal labels, we train one policy per cost weight λ with REINFORCE on reward $r = \text{correct}(a) - \lambda \cdot \text{cost}(a)/\text{cost}_{\text{ref}}$, using only the sampled arm's outcome. On a held-out test set of 189 benchmark and agent-style prompts, the policy selected on validation for the default `auto` mode ($\lambda = 0$) reaches 83.1% accuracy (95% CI 77.2–88.4) at 86.2% of the cost of always calling the largest model (CI 78.8–92.4), versus 84.1% for always-large. These are offline results on a small test set with one seed; we report confidence intervals throughout and compare against strong cheap baselines.

DRAFT · PARTIAL RL RESULTS

1. Introduction

Coding agents and LLM applications route nearly every call, from classification and summaries to multi-step reasoning, to one expensive model. Teams that measure AI spend in *dollars per merged pull request* pay frontier prices for requests a small open model would get right. LLM routing addresses this by choosing a model per request [2], [3], [4].

A recent class of *decision models* returns typed answers with probabilities instead of generating text: the closed Jev [5] and the open-weight Laya [1]. We observe that such a model's output over a small, typed option set is exactly a stochastic policy, so it can be trained directly with policy gradients without token-level RL.

Contributions (scoped to what we measured):

1. A router that uses Laya's option distribution as the policy $\pi(a|s)$ over three model tiers, trained with a supervised warm start followed by REINFORCE on a correctness-minus-cost reward, yielding a family of policies indexed by λ.
2. An evaluation on 948 automatically graded prompts (math, science QA, code, and an agent-style slice of bug-fix and CI-triage tasks built by bug injection), with bootstrap confidence intervals and cheap embedding baselines, including per-tier correctness predictors swept over the same λ grid.
3. An open implementation: an OpenAI-compatible proxy with sticky per-session routing and tool-call passthrough, so agent CLIs can use the router by changing one base URL.

2. Related Work

LLM routing and cascades. RouteLLM trains routers from preference data to choose between a strong and a weak model [2]. FrugalGPT learns cascades over multiple LLMs to reduce cost [3]. AutoMix combines small-model self-verification with a POMDP-based router that decides when to escalate [4]. RouterBench provides a benchmark and a large dataset of inference outcomes for evaluating routers [6]; we have not evaluated on it yet.

Decision models. Jev is a closed decision model returning typed choices, scores and yes/no answers with probabilities [5]. Laya is an open-weight model of the same category: a ModernBERT-large encoder [7] with a decision head that scores each option at its own marker in one pass [1]. REFLEX uses Jev as a fast typed decision layer inside an agent and escalates to a strong LLM when confidence is low [8]. Our work differs in *training* the decision model as a routing policy with RL on observed outcomes, using open weights end to end.

3. Method

3.1. Decision-model output as a policy

Laya encodes `[CLS] choice question: <instructions> [SEP] [MASK] opt₀ [MASK] opt₁ [MASK] opt₂ [SEP] <request> [SEP]` and reads one logit per option at its `[MASK]` position [1]. With three options (`small`, `medium` and `large`, each given a short description) the softmax over these logits defines $\pi_{\theta(a|s)}$ for state s = the request text. One forward pass yields the full distribution, so $\log \pi_{\theta(a|s)}$ is available for every action at the cost of a single encoder call.

3.2. Reward

Each prompt was answered by all three tiers and graded automatically (Section 4). For action a, $r(s,a) = \mathbb{1}[\text{correct}(s,a)] - \lambda \cdot c(s,a)/c_{\text{ref}}$, where $c(s,a)$ is the measured USD cost of that call and c_{ref} = 0.003138 USD is the mean cost of always-large on the training split, which makes λ unitless.

3.3. Supervised warm start

We first fine-tune on the *oracle* label (the cheapest tier that answered correctly, or `large` if none did) with cross-entropy over the three options, for up to 10 epochs with early stopping on validation loss. To fit a 4 GB GPU we train only the top 4 encoder layers plus Laya's decision head. Because the lower layers are frozen, their outputs are computed once per prompt and cached, and training runs only the trainable layers (the result is identical to the full forward up to bf16 rounding).

3.4. REINFORCE with baseline and entropy

Starting from the warm start, for each $\lambda \in \{0, 0.1, 0.25, 0.5, 1, 2\}$ we train a separate policy for 800 steps with batches of 16 training prompts. For each prompt we sample $a \sim \pi_{\theta(\cdot|s)}$, look up only that arm's reward, and minimise

$$\mathcal{L} = -(r - b) \log \pi_{\theta(a|s)} - \beta H\left(\pi_{\theta(\cdot|s)}\right)$$

with b an exponential running mean of the batch reward [9] and β annealed linearly from 0.05 to 0.005. Using only the sampled arm simulates the online partial-feedback setting, even though the offline table contains all three outcomes. The checkpoint with the highest validation reward is kept; the test split is never used for selection. The three proxy modes (`auto`, `auto-quality`, `auto-cheap`) map to trained λ values through fixed rules applied to validation results.

4. Experimental Setup

Data. 948 prompts with automatically checkable answers, mixing easy and hard sources because a pilot showed current open models saturate the easy ones: GSM8K [10] (final number, exact match), MATH-500 levels 4–5 (the 500-problem subset [11] of MATH [12]) (last `\boxed{}` answer after LaTeX normalisation), ARC-Challenge [13] and MMLU-Pro [14] (option letter), MBPP [15] (unit tests in

DRAFT · PARTIAL RESULTS

a sandboxed subprocess, 5 s timeout), and an agent-style slice built from disjoint, longer MBPP functions: *bug-fix* prompts (one or two injected bugs among off-by-one, wrong operator, inverted condition and wrong return; graded by the original tests) and *CI-triage* prompts (classify the injected bug type from the code and real test output; graded against the label known from construction). Splits are stratified by source, 569 / 190 / 189 (train / val / test).

Model tiers. All three tiers are open-weight models with tool calling, served through OpenAI-compatible endpoints. Prices are per million tokens (input / output, USD):

Tier	Model	In	Out
small	nvidia/nemotron-3.5-bolt-e4b	0.04	0.05
medium	openai/gpt-oss-20b	0.018	0.09
large	nvidia/nemotron-3-ultra-550b-a55b	0.6	2.4

All calls ran on NVIDIA's free API (actual spend $0), so every cost is computed at the public list price of the same model on a paid provider (OpenRouter). The small tier has no paid listing; its price is a *proxy price*, the list price of `google/gemma-3-4b-it`, a model of the same ~4B size class.

Baselines. Always-small/medium/large; uniform random; the oracle (hindsight, not achievable); zero-shot Laya; logistic regression and k-NN predicting the oracle tier from frozen embeddings (base Laya, mean-pooled; and all-MiniLM-L6-v2 [16]), with C and k tuned on validation; and per-tier correctness predictors, one logistic regression per tier estimating $P(\text{correct}|s,a)$, acting by $\arg\max_a P(\text{correct}|s,a) - \lambda \cdot \bar{c}_a / c_{\text{ref}}$ over the same λ grid. Jev: not run (no JEV_API_KEY).

Metrics. Test accuracy, mean cost per request, cost relative to always-large, and router latency, each with 95% percentile bootstrap intervals (1000 resamples). Comparisons against always-large use paired bootstrap differences.

Training. SFT learning rate 2e-05 RL learning rate 5e-06, AdamW, gradient clipping 1.0, bf16 autocast, seed 0. Hardware: a single 4 GB laptop GPU (RTX 3050).

DRAFT PARTIAL REF-SUITES

5. Results

Policy	Accuracy % [95% CI]	Cost % of always-large [95% CI]	Router ms	Pareto
always-small	73.5 [67.2, 79.9]	1.0 [0.8, 1.1]	0.0	✓
always-medium	82.5 [77.2, 87.8]	2.1 [1.8, 2.3]	0.0	✓
always-large	84.1 [78.8, 88.9]	100.0 [100.0, 100.0]	0.0	
random	79.9 [74.1, 85.2]	41.4 [32.4, 51.5]	0.0	
oracle	93.7 [90.5, 96.8]	26.0 [17.4, 35.1]		
zero-shot Laya	77.2 [71.4, 83.1]	20.1 [12.4, 28.7]	43.9	
LR (Laya-emb)	74.1 [68.3, 79.9]	6.8 [2.6, 12.4]	30.9	
kNN (Laya-emb)	73.5 [67.2, 79.9]	1.1 [0.9, 1.5]	30.9	
per-tier LR (Laya-emb) (λ=0.1)	82.5 [76.7, 87.8]	8.5 [4.2, 13.4]	30.9	
per-tier LR (Laya-emb) (λ=0.25)	81.5 [75.7, 86.8]	2.8 [1.7, 4.5]	30.9	
per-tier LR (Laya-emb) (λ=0.5)	81.5 [75.7, 86.8]	1.8 [1.5, 2.0]	30.9	✓
per-tier LR (Laya-emb) (λ=1)	81.0 [75.1, 86.2]	1.8 [1.5, 2.0]	30.9	✓
LR (MiniLM-emb)	75.1 [68.8, 81.0]	4.8 [1.5, 9.0]	6.4	
kNN (MiniLM-emb)	73.5 [67.2, 79.9]	1.1 [0.9, 1.3]	6.4	
per-tier LR (MiniLM-emb) (λ=0.1)	83.1 [77.8, 88.4]	7.6 [3.9, 12.3]	6.4	✓
per-tier LR (MiniLM-emb) (λ=0.25)	82.0 [76.7, 87.3]	3.4 [2.0, 5.1]	6.4	
per-tier LR (MiniLM-emb) (λ=0.5)	81.0 [75.1, 86.2]	1.9 [1.6, 2.1]	6.4	
per-tier LR (MiniLM-emb) (λ=1)	81.0 [75.1, 86.2]	1.8 [1.6, 2.1]	6.4	
SFT Laya	73.5 [67.2, 79.9]	1.0 [0.8, 1.1]	43.9	✓
RL Laya (λ=0)	83.1 [77.2, 88.4]	86.2 [78.8, 92.4]	43.9	

Table 1: Test-set results (n = 189). Per-tier LR rows are shown for a subset of λ. Router latency is the median single-request time on the build machine's GPU.

DRAFT • PARTIAL RESULTS

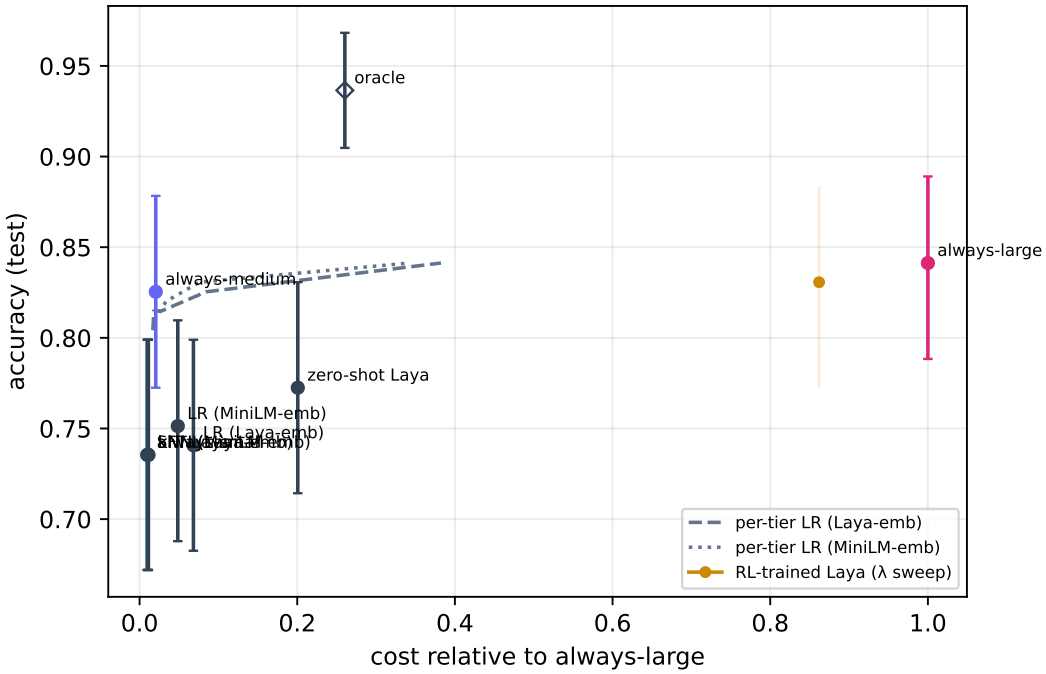


Figure 1: Accuracy vs. cost relative to always-large. Shaded band: 95% CI of the RL family.

The validation-selected `auto` policy ($\lambda = 0$) differs from always-large in accuracy by -1.1 points (paired 95% CI -3.7 to +1.6) while costing 86.2% as much. Zero-shot Laya reaches 77.2% at 20.1% cost. The supervised warm start raises validation agreement with the oracle label from 32.6% (zero-shot) to 69.5% (epoch 10). Table 1 and Figure 1 place the RL family against the per-tier correctness predictors, the strongest cheap baseline. Where those match or beat the RL policies, Pennywise's remaining advantages are architectural (a single model scoring options directly, trainable from bandit feedback) rather than accuracy.

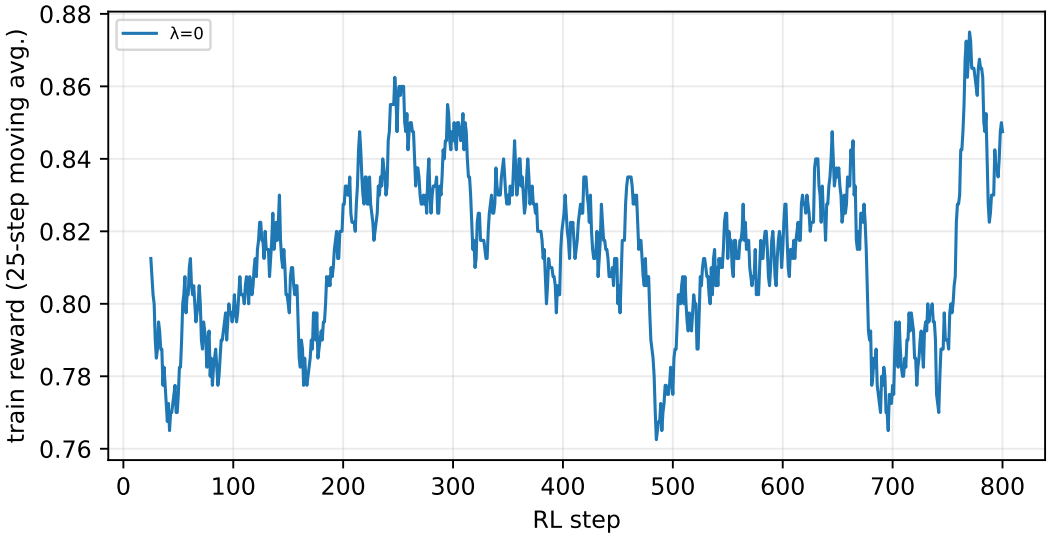


Figure 2: Training reward (sampled actions, 25-step moving average) per λ.

Source	always-small	always-medium	always-large	RL Laya (λ=0)
Agent bug-fix (n=19)	68.4 / 0.3	78.9 / 1.7	78.9 / 100.0	84.2 / 82.9
Agent CI triage (n=20)	60.0 / 0.6	80.0 / 0.9	80.0 / 100.0	70.0 / 76.6
ARC-Challenge (n=20)	85.0 / 0.7	100.0 / 1.1	95.0 / 100.0	95.0 / 100.0
GSM8K (n=20)	95.0 / 1.1	95.0 / 2.2	90.0 / 100.0	90.0 / 100.0
MATH L4-5 (n=30)	53.3 / 1.7	70.0 / 3.0	70.0 / 100.0	70.0 / 93.3
MBPP (n=40)	82.5 / 0.7	97.5 / 2.1	90.0 / 100.0	87.5 / 78.7
MMLU-Pro (n=40)	72.5 / 1.0	65.0 / 1.9	85.0 / 100.0	85.0 / 84.5

Table 2: Per-source accuracy % / cost % of always-large on the test split. The agent-style slices are reported separately.

Router latency. The router adds one encoder pass: 43.9 ms median on the build machine's GPU for `auto`. CPU latency on the deployment host is reported in the repository.

6. Limitations and Future Work

The test set is small (n = 189), so intervals are wide. We use a single seed. The setting is offline: although RL uses only the sampled arm, the outcome table was collected with full information, which an online system would not have. Benchmark prompts are not production traffic, and the agent slice consists of synthetic single-function tasks. Laya is known to degrade with many options [1], which is irrelevant at three tiers but limits extension to many models. Jev was not benchmarked.

Planned work: partial-feedback online RL with a feedback endpoint; comparison with an exact expected-reward objective (possible offline because all outcomes are known); evaluation on RouterBench [6]; out-of-distribution tests; more tiers; multiple seeds; calibration analysis.

7. Reproducibility

Code, configs and this report's sources are at the project repository, commit `a0644e5-dirty`. `config/models.yaml` (tiers, prices) and `config/train.yaml` (splits, hyperparameters, λ grid, seed) define the run. Commands: `python data/build_prompts.py`, `python data/run_models.py`, `python -m eval.features`, `python -m eval.baselines`, `python -m router.sft`, `python -m router.rl`, `python -m eval.evaluate`, `python -m eval.export_viz`, `python paper/build.py`.

Bibliography

[1] Convai Innovations, "Laya: model card." [Online]. Available: https://huggingface.co/convaiinnovations/laya

[2] I. Ong *et al.*, "RouteLLM: Learning to Route LLMs with Preference Data." [Online]. Available: https://arxiv.org/abs/2406.18665

[3] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance." [Online]. Available: https://arxiv.org/abs/2305.05176

[4] P. Aggarwal *et al.*, "AutoMix: Automatically Mixing Language Models." [Online]. Available: https://arxiv.org/abs/2310.12963

[5] LDS Team, "TypeSafe AI Launches Jev for Structured Software Decisions." [Online]. Available: https://letsdatascience.com/news/typesafe-ai-launches-jev-decision-model-889a38c0

[6] Q. J. Hu *et al.*, "RouterBench: A Benchmark for Multi-LLM Routing System." [Online]. Available: https://arxiv.org/abs/2403.12031

DRAFT · PARTIAL RESULTS

[7] B. Warner *et al.*, "Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference." [Online]. Available: https://arxiv.org/abs/2412.13663

[8] T. Wu and W. Y. B. Lim, "REFLEX with Jev for Efficient Selective Control in LLM Agents." [Online]. Available: https://arxiv.org/abs/2609.26532

[9] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992, doi: 10.1007/BF00992696.

[10] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems." [Online]. Available: https://arxiv.org/abs/2110.14168

[11] H. Lightman *et al.*, "Let's Verify Step by Step." [Online]. Available: https://arxiv.org/abs/2305.20050

[12] D. Hendrycks *et al.*, "Measuring Mathematical Problem Solving With the MATH Dataset." [Online]. Available: https://arxiv.org/abs/2103.03874

[13] P. Clark *et al.*, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge." [Online]. Available: https://arxiv.org/abs/1803.05457

[14] Y. Wang *et al.*, "MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark." [Online]. Available: https://arxiv.org/abs/2406.01574

[15] J. Austin *et al.*, "Program Synthesis with Large Language Models." [Online]. Available: https://arxiv.org/abs/2108.07732

[16] sentence-transformers, "all-MiniLM-L6-v2: model card." [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

DRAFT · PARTIAL RESULTS